Software Requirements Specification (SRS)

For Student Elective Registration System

Version <1.0> - Draft

Prepared by:
Group Name: NOVA

Ananya Agrawal	se23uari010@mahindrauniversity.edu.in
H. Aakanksh Reddy	se23uari048@mahindrauniversity.edu.in
Aishika Reddy	se23uari075@mahindrauniversity.edu.in
Shruti Verma	se23uari116@mahindrauniversity.edu.in
N. Sudhiksha	se23ucse119@mahindrauniversity.edu.in
P. Sishir K. Reddy	se23ucse136@mahindrauniversity.edu.in
S. Nayanasakhi	se23umcs056@mahindrauniversity.edu.in

Instructor:	Murali Krishna
Course:	Software Engineering
Lab Section:	*IT, Wing-3, FF, CR1*
Teaching Assistant:	*NA*
Date:	Tentative

Contents

CONTENTS II

REVISIONS III

1 INTRODUCTION 1

1.1 DOCUMENT PURPOSE 1
1.2 PRODUCT SCOPE 1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW 1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS 2
1.5 DOCUMENT CONVENTIONS 2
1.6 REFERENCES AND ACKNOWLEDGMENTS 3

2 OVERALL DESCRIPTION 4

2.1 PRODUCT OVERVIEW 4
2.2 PRODUCT FUNCTIONALITY 5
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS 5
2.4 ASSUMPTIONS AND DEPENDENCIES 6

3 SPECIFIC REQUIREMENTS 7

3.1 EXTERNAL INTERFACE REQUIREMENTS 7
3.2 FUNCTIONAL REQUIREMENTS 8
3.3 USE CASE MODEL 10

4 OTHER NON-FUNCTIONAL REQUIREMENTS 13

4.1 PERFORMANCE REQUIREMENTS 13
4.2 SAFETY AND SECURITY REQUIREMENTS 13
4.3 SOFTWARE QUALITY ATTRIBUTES 13

5 OTHER REQUIREMENTS 14

APPENDIX A – DATA DICTIONARY 15

APPENDIX B - GROUP LOG 16

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
V-1	Ananya Agrawal N. Sudhiksha H. Aakanksh R. P. Sishir K. R. Shruti Verma S. Nayanasakhi Aishika Reddy	Established project scope and 3-tier system architecture (React/Spring Boot/PostgreSQL) for a web-based elective registration system. Defined core modules for visual timetable scheduling, automated prerequisite validation, conflict detection, and a weighted CGPA + FCFS allocation algorithm.	15/03/2026

1 Introduction

The **Student Elective Registration System (SERS)** is a web-based platform designed to streamline the elective course selection and allocation process for students in the Engineering School. The system allows students to view eligible electives, submit ranked preferences, and receive course allocations, while administrators can manage course offerings, configure seat capacities, and oversee the allocation process. By digitizing and structuring the registration workflow, the system aims to improve clarity, efficiency, and fairness in elective selection.

This section introduces the project and provides an overview of the Software Requirements Specification (SRS) document. It outlines the scope of the system, identifies the intended audience of the document, and explains the definitions, acronyms, and document conventions used throughout the specification. It also lists references and supporting materials relevant to the system requirements.

1.1 Document Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of the SERS. It provides a detailed description of the system's expected behavior, system constraints, and interactions between users and the application. The document serves as a reference for the design, development, and testing of the system.

This SRS primarily focuses on the core elective registration functionalities, including course management, student preference submission, and the allocation process managed by administrators. It outlines the system requirements necessary to support these features and provides a structured foundation for implementing and evaluating the software throughout the project lifecycle.

1.2 Product Scope

The SERS is a web-based application designed to support the elective registration process for students in the Engineering School. The system focuses on third- and fourth-year students, who must select the required number of branch-specific electives and humanities electives. Branch electives are restricted to courses offered within a student's own branch, while humanities electives are common and available to students across multiple branches. For the scope of this project, the system considers approximately four to five engineering branches.

The system allows students to view eligible electives, submit ranked course preferences, and view their final allocations. Administrators can manage course offerings, configure seat capacities, and run the allocation process. By centralizing these functions, the system aims to simplify elective registration, reduce confusion during course selection, and ensure that eligibility rules and seat limits are enforced consistently.

The terminology used in this document, such as student, administrator, allocation, and preference submission, follows the definitions provided in Section 1.4.

1.3 Intended Audience and Document Overview

This document is intended for stakeholders involved in the design, development, and evaluation of the SERS. The primary readers include the professor, who reviews the system requirements and design decisions, and the client or academic stakeholders, who evaluate whether the system meets the intended needs of the elective registration process. It is also intended for developers and testers who

will use the documented requirements and use cases as a reference during system design, implementation, and testing.

The introduction section provides background information, scope, and definitions used throughout the document. Subsequent sections describe the overall system description, functional and non-functional requirements, and system interactions. Readers are encouraged to begin with the Introduction and Product Scope sections to understand the context of the project and then proceed to the system description and requirements sections, which provide detailed specifications necessary for development and evaluation.

1.4 Definitions, Acronyms and Abbreviations

This section defines the terms, acronyms, and abbreviations used throughout this document to ensure a common understanding among all readers.

Term / Acronym	***Definition***
Administrator (Admin)	A user responsible for managing courses, configuring seat capacities, and initiating the allocation process.
Allocation	The process of assigning students to elective courses based on preferences and system rules.
Branch Elective	An elective course restricted to students belonging to a specific branch.
COMET	It's a lifecycle methodology for designing concurrent, real-time, and distributed systems by integrating OOPS with architectural modeling.
Database	The PostgreSQL system used to store persistent data including students, courses, preferences, and allocations.
Elective Course	A course selected by students from a list of available options rather than being a mandatory course.
ESLint	A tool that scans your code to catch errors and enforce a consistent coding style automatically.
Humanities Elective	A non-technical elective course that is open to students across multiple engineering branches.
Priority Score	A calculated score used by the system to rank students during the allocation process, derived from CGPA, year of study, and preference submission timestamp.
RBAC	It's a security model that restricts system access to authorized users based on their assigned professional roles rather than their individual identities.
REST API	Representational State Transfer Application Programming Interface used for communication between the frontend and backend components of the system.
SERS	Student Elective Registration System, the web-based system described in this document.
XSS	(Cross-Site Scripting) is a vulnerability where attackers inject malicious scripts into webpages to steal user data.

1.5 Document Conventions

This document follows the **IEEE Software Requirements Specification (SRS) guidelines defined in IEEE 29148**, which provide a structured approach for specifying system requirements.

Requirement statements in this document use standard requirement terminology. The keyword **"shall"** indicates mandatory system behavior, **"should"** indicates recommended behavior, and **"may"** indicates optional functionality. Each requirement is assigned a unique identifier to support traceability throughout the document:

- Functional requirements are labeled using identifiers such as **F1, F2, F3**.
- Use cases are labeled using identifiers such as **U1, U2, U3**.
- Non-functional requirements use category identifiers such as **P** (Performance), **S** (Security), and **Q** (Quality Attributes).

Section headings follow the hierarchical numbering structure of the IEEE SRS template (e.g., 1, 1.1, 1.2). Lists and tables are used where appropriate to improve clarity and readability.

1.6 References and Acknowledgments

The following resources were referenced during the preparation of this Software Requirements Specification:

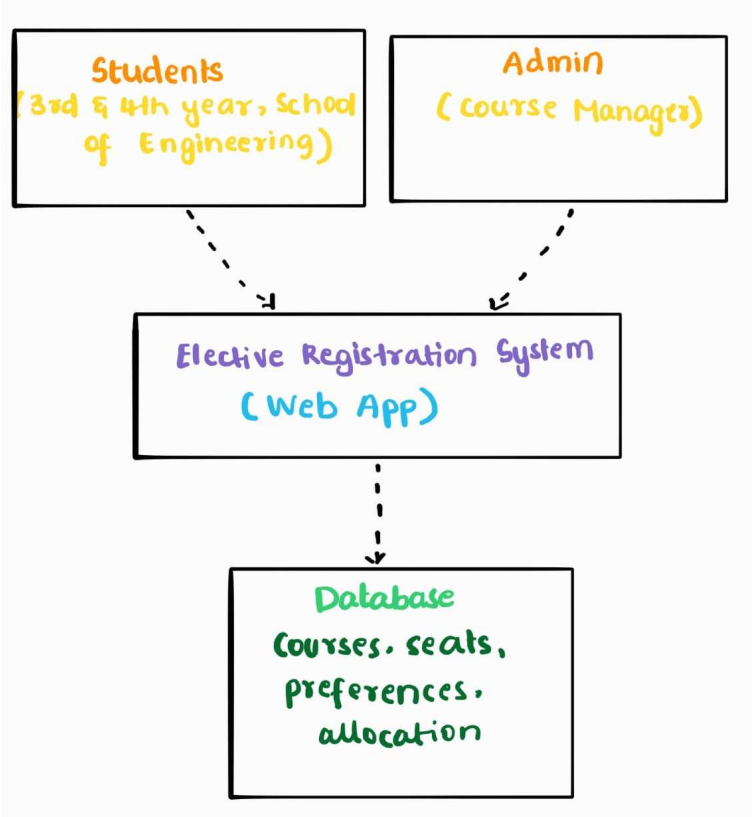
1. IEEE. *IEEE 29148-2018 – Systems and Software Engineering: Life Cycle Processes – Requirements Engineering.*
2. Spring Boot Documentation – https://spring.io/projects/spring-boot
3. React Documentation – https://react.dev
4. PostgreSQL Documentation – https://www.postgresql.org/docs/

2 Overall Description

2.1 Product Overview

The SERS is designed as a replacement for the current elective selection process used within Mahindra University. Existing processes often involve manual coordination, limited visibility of available electives, and difficulty managing seat capacities and course eligibility. The proposed system introduces a centralized web-based platform that allows students to view eligible electives, submit course preferences, and receive final allocations, while enabling administrators to manage course offerings and control the allocation process more efficiently.

This system operates as a self-contained web application within the academic environment of the School of Engineering in Mahindra University. It interacts primarily with two types of users: students, who submit elective preferences and view results, and administrators, who configure courses, set seat limits, and run the allocation process. The system stores and manages all relevant data such as course information, student preferences, and allocation results within a database. The overall interaction between users, the application, and the database can be illustrated in the high-level diagram below.



High Level Diagram

2.2 Product Functionality

The SERS provides the following major functions:

1. Allow students to view eligible branch and humanities electives based on their academic program.
2. Enable students to submit ranked preferences for elective courses.
3. Allow students to view their final elective allocations after the allocation process is completed.
4. Enable administrators to create and manage elective courses, including branch-specific and humanities electives.
5. Allow administrators to configure course details, such as seat capacity and course availability.
6. Provide administrators with the ability to initiate and manage the elective allocation process.
7. Maintain a centralized database of students, courses, preferences, and allocation results.
8. Ensure that course eligibility rules and seat capacity constraints are enforced during the allocation process.

The detailed system behavior associated with these capabilities is formally defined in the Functional Requirements described in Section 3.2.

2.3 Design and Implementation Constraints

The following constraints dictate the boundaries within which the SERS must be developed:

- **Technology & Language:** The system must be built using React for the frontend and Spring Boot (Java) for the backend. Data persistence must utilize PostgreSQL to handle complex relational queries and concurrent transactions.

- **Hardware & Performance:** To meet the 2–3 second response time requirement for course lists, the architecture must support low-latency data retrieval. Developers must optimize database indexing and implement real-time seat availability updates to prevent over-subscription during peak loads.

- **Parallel Operations:** The design must handle high-volume concurrent registration attempts. Developers must use database transaction management and "pessimistic" or "optimistic" locking to ensure data consistency when multiple students select the same course simultaneously.

- **Interfaces & Protocols:** Communication must be restricted to RESTful APIs over HTTPS. The system must interface with existing University Authentication Services to facilitate secure university login credentials.

- **Security & Safety:** Constraints include mandatory Role-Based Access Control (RBAC) to isolate student data from administrative functions. To prevent data tampering and accidental loss, the design must include automated database backups and input sanitization to block SQL injection and XSS.

- **Design Standards:** The system will follow established programming standards such as Google Java Style for backend code and ESLint guidelines for frontend JavaScript to maintain code consistency and ease of maintenance.

- **Methodology:** Development must follow the COMET (Concurrent Object Modeling and Architectural Design Method) to address the system's concurrency requirements. All system views must be documented using UML modeling.

References

Gomaa, H. (2000). Designing Concurrent, Distributed, and Real-Time Applications with UML. Addison-Wesley. (Primary reference for the COMET method).

Object Management Group (OMG). (2017). Unified Modeling Language (UML) Specification Version 2.5.1. https://www.omg.org/spec/UML/

2.4 Assumptions and Dependencies

1. The timetable, faculty assignments, class days, and time slots are predefined and will not change during the registration period.
2. The system follows Mahindra University's existing academic structure, scheduling format, and elective policies.
3. Course capacities, prerequisites, and eligibility rules are correctly defined before the registration process begins.
4. Student academic records and branch information used by the system are accurate and up to date.
5. Students are only shown courses they are eligible for based on year, branch, prerequisites, and timetable constraints, preventing invalid course selections.
6. The application relies on the availability of the database system and web server infrastructure to store and retrieve course, student, and allocation data.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The SERS provides a web-based user interface that enables students and administrators to interact with the system through a standard web browser. The interface is designed to provide clear visibility into elective options, preference submission, and allocation outcomes. The system provides the following different interface views depending on the role of the user:

Student Interface- This interface allows students to interact with the system. The main components include:

- **Student Dashboard:** Displays student details such as branch, year of study, and registration status.
- **Elective Browsing Interface:** Shows the list of eligible branch electives and humanities electives along with course information.
- **Preference Submission Interface:** Allows students to select and rank three branch electives and one humanities elective.
- **Allocation Result Interface:** Displays the final course allocated to the student after the allocation process is completed.

Input interactions are primarily performed through forms, selection menus, and submission buttons. Validation messages are displayed to notify users of invalid inputs (conflicts) or incomplete preference selections.

Administrator Interface- This interface enables authorized academic staff to manage the elective registration process. Key components include:

- **Course Management Interface:** Allows administrators to create, edit, activate, or deactivate elective courses. Basically can make changes.
- **Seat Configuration Interface:** Enables administrators to define seat capacities for each course.
- **Allocation Control Interface:** Allows administrators to trigger the elective allocation process.

3.1.2 Hardware Interfaces

The SERS operates as a website-based application and does not require specialized hardware interfaces. Users interact with the system using standard computing devices such as:

- Desktop / Laptop computers
- Tablets or mobile devices with web browsers

Input is provided through standard input devices including keyboards, touchscreens, and pointing devices. The system requires internet connectivity for communication with the backend server.

3.1.3 Software Interfaces

SERS interacts with several software components that together enable system functionality. They are as follows:

1. **Frontend Application-** The frontend of the system is implemented using **React**, which provides the presentation layer of the application. It is responsible for rendering the user interface, capturing user inputs, and sending requests to the backend services through REST APIs.
2. **Backend Application-** The backend is implemented using **Spring Boot** and contains the core business logic of the system. It processes user requests, performs validation checks, executes the elective allocation algorithm, and communicates with the database layer.
3. **Database System-** The system uses **PostgreSQL** as its database management system. The database stores persistent data including:
    - Student information
    - Elective course details
    - Submitted preferences
    - Allocation results
    - Seat capacity configurations

Communication between the frontend and backend occurs through REST API endpoints, while the backend interacts with PostgreSQL through database queries and repository interfaces.

3.2 Requirements

3.2.1 Functional Requirements

The following functional requirements define the expected behavior of the Student Elective Registration System (SERS):

F1- Student Authentication: The system shall allow students to securely log into the SERS using their institutional credentials before accessing elective registration features.

F2- Display Eligible Electives: The system shall display a list of electives that are eligible for the student based on the student's branch and year of study.

F3- Display Course Details: The system shall provide course information for each elective, including course title, course code, course category (branch elective or humanities elective), and seat capacity.

F4- Preference Submission: The system shall allow students to submit ranked preferences consisting of three branch electives and one humanities elective.

F5- Preference Validation:

The system shall validate submitted preferences to ensure that:

- only eligible courses are selected
- duplicate courses are not selected
- all required preference slots are filled

Invalid submissions shall be rejected with an appropriate error message.

F6- Store Student Preferences: The system shall store submitted student preferences in the database along with the submission timestamp.

F7- Course Management: The system shall allow administrators to create and manage elective courses, including defining course type, availability, and seat capacity.

F8- Seat Capacity Configuration: The system shall allow administrators to define and update seat capacities for each elective course.

F9- Allocation Execution: The system shall allow an administrator to initiate the elective allocation process after preference submission has been completed.

F10- Priority Score Calculation: The system shall calculate a Priority Score for each student during allocation. The score shall be determined using the student's CGPA, year of study, and preference submission timestamp.

F11- Student Ranking: The system shall rank students according to their calculated Priority Score before assigning elective seats.

F12- Seat Allocation: The system shall allocate elective seats to students based on their ranked preferences and available seat capacity.

F13- Capacity Enforcement: The system shall ensure that the number of students allocated to a course does not exceed the defined seat capacity.

F14- Store Allocation Results: The system shall store final allocation results in the database after the allocation process is completed.

F15- View Allocation Results: The system shall allow students to view their allocated elective course through the system interface.

3.2.2 Non-functional Requirements

N1 – Response Time: The system shall respond to user interface requests (such as viewing electives or submitting preferences) within 2 seconds under normal operating conditions.

N2 – Peak Load Handling: The system shall support at least 500 concurrent student users during peak elective registration periods without system failure.

N3 – System Availability: The system shall maintain 99% uptime availability during the elective registration window.

N4 – Data Integrity: The system shall ensure that stored preference and allocation data cannot be altered without authorized administrative access.

N5 – Secure Communication: All communication between the client interface and the server shall be encrypted using HTTPS/TLS protocols.

N6 – Fault Recovery: The system shall automatically recover and restore normal operation within 5 minutes after a system failure or restart.

N7 – Audit Logging: The system shall record logs of critical actions such as preference submission, course creation, and allocation execution for administrative review.

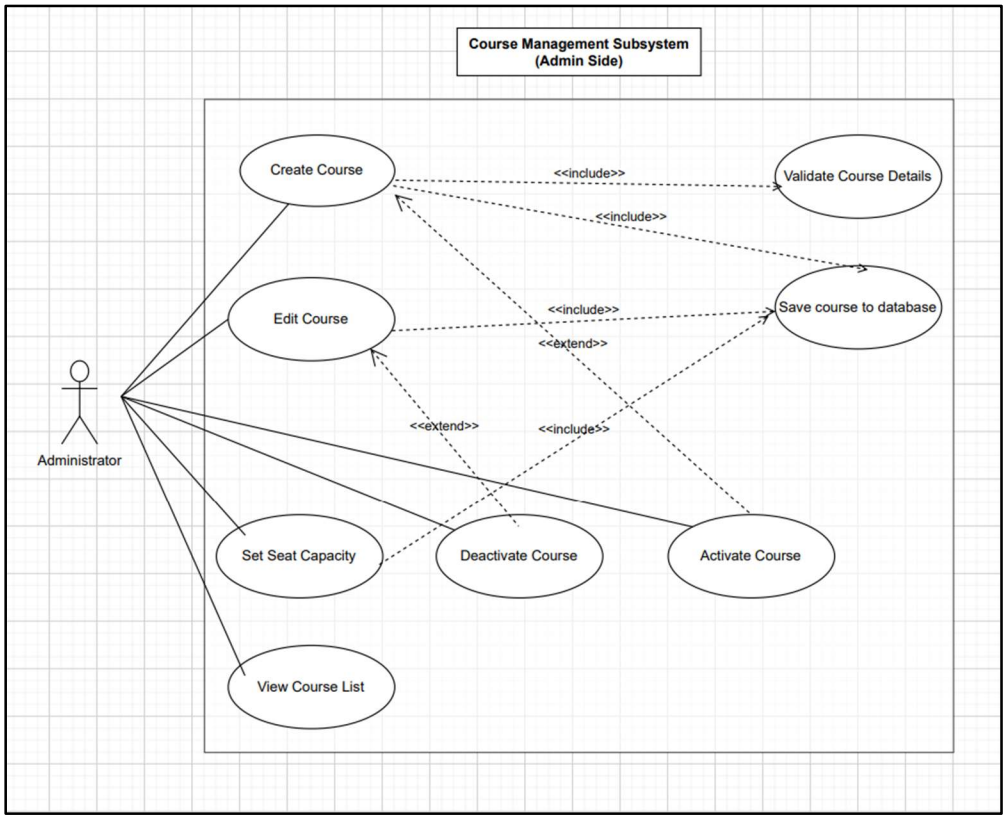
N8 – Maintainability: The system shall follow standardized coding conventions and modular architecture to enable future modifications without significant restructuring.

N9 – Scalability: The system shall support expansion to accommodate additional academic branches and increased student enrollment without requiring fundamental system redesign.

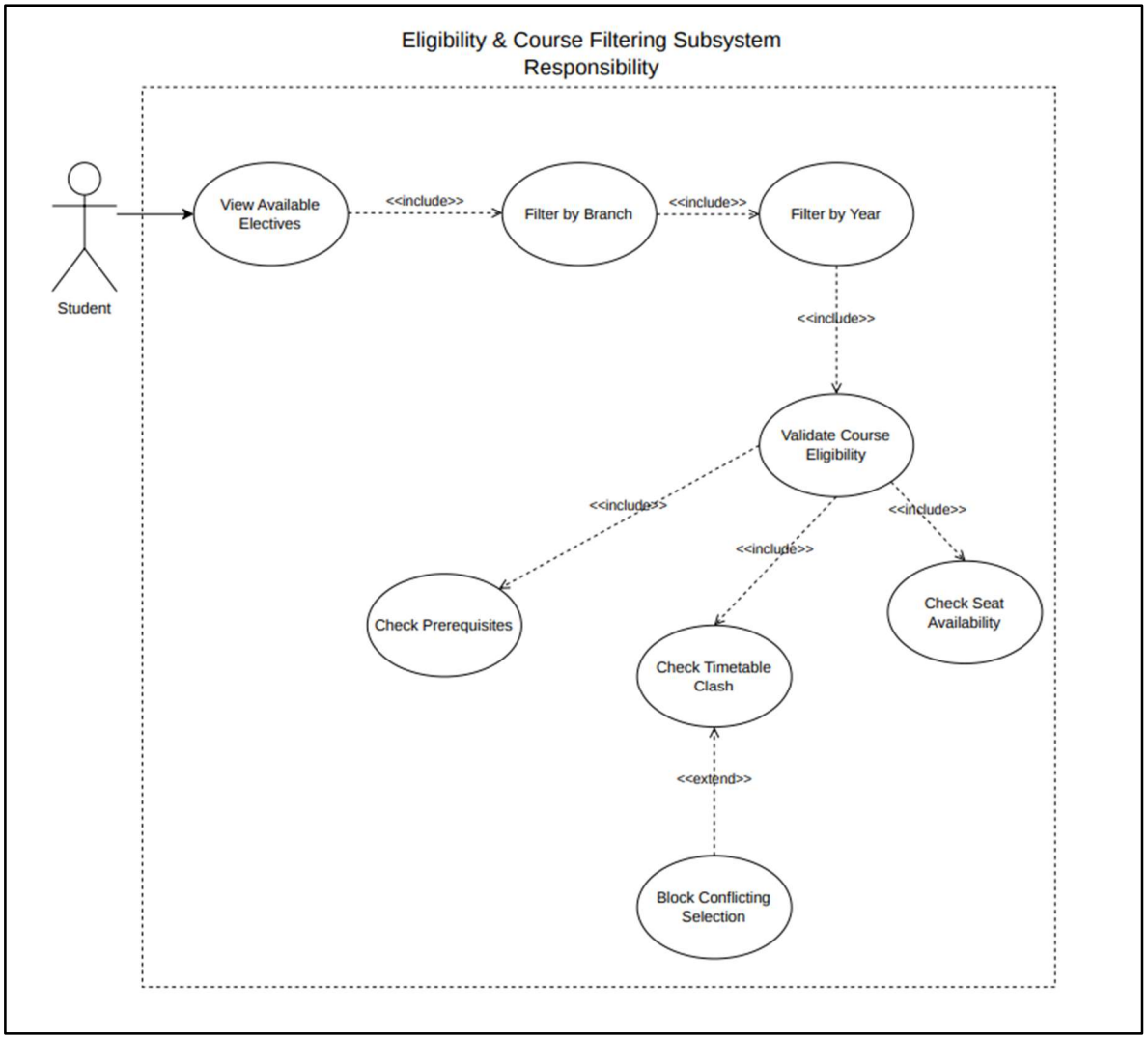
N10 – Data Backup: The system shall maintain periodic backups of course data, student preferences, and allocation results to prevent permanent data loss.

3.3 Use Case Model

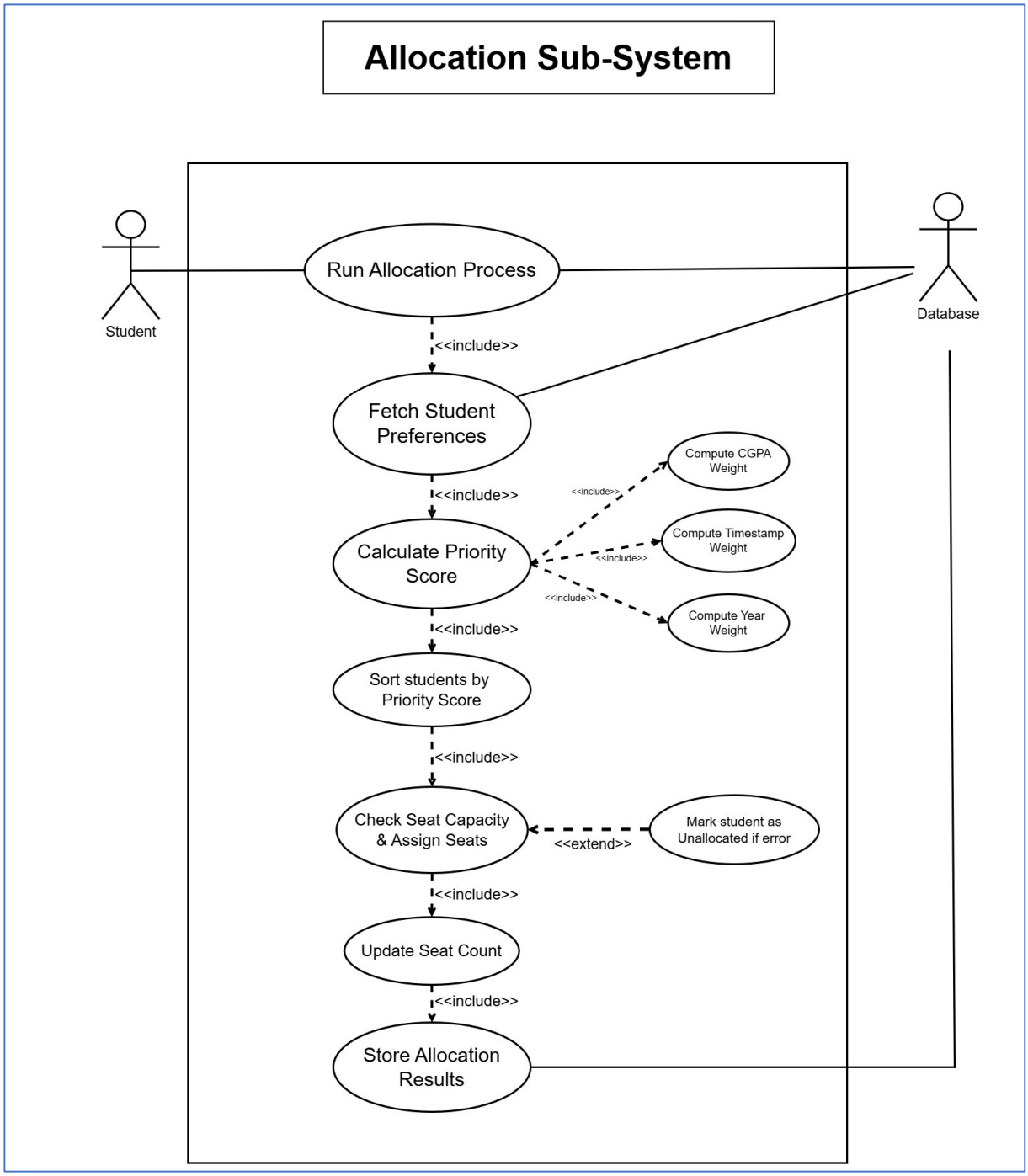
3.3.1 Use Case 1:



3.3.2 Use Case 2:



3.3.3 Use Case 3:



4 Other Non-functional Requirements

4.1 Performance Requirements

The Elective Course Registration System should provide fast and efficient responses to user actions, particularly during peak course registration periods when many students access the system simultaneously. The system should be capable of processing multiple registration requests at the same time without significant delays.

The system should display the list of available elective courses within **2–3 seconds** after a student requests it. Course registration confirmation should also be processed immediately after the student selects a course. The system must maintain real-time updates of course availability to ensure that students cannot register for courses that have reached maximum capacity.

4.2 Safety and Security Requirements

The Elective Course Registration System must ensure the security and confidentiality of user information. Only authorized users should be able to access the system through secure authentication mechanisms such as university login credentials.

Students should only have access to their own course registration data, while administrative privileges such as adding or modifying courses should be restricted to authorized administrators. The system must protect sensitive information including student details, course registrations, and login credentials. Additionally, the system should prevent unauthorized access, data tampering, and accidental loss of information by implementing appropriate security measures and access controls.

4.3 Software Quality Attributes

4.3.1 Reliability

The system should operate reliably during elective course registration periods and ensure that all student registrations are accurately recorded. In case of unexpected system failures, the system should recover quickly without losing important registration data.

4.3.2 Usability

The system should provide a simple and intuitive user interface that allows students to easily browse available elective courses, view course details, and complete the registration process without confusion.

4.3.3 Maintainability

The system should be designed in a modular and well-structured manner so that future updates, bug fixes, and feature improvements can be implemented efficiently without affecting other components of the system.

4.3.4 Scalability

The system should be able to support an increasing number of students, courses, and registrations in future academic sessions without significant performance degradation.

5 Other Requirements

This section specifies additional requirements for the Elective Course Registration System that are not covered in the previous sections of the SRS. These requirements ensure that the system operates effectively within the university environment while supporting scalability, maintainability, and compliance with institutional policies.

5.1 Database Requirements

The system will use a centralized database to store and manage information related to students, courses, instructors, and course registrations. The database must ensure data consistency, integrity, and accuracy. It should maintain updated records of course availability, student enrollments, and registration history. Regular database backups should be performed to prevent data loss.

5.2 Legal and Institutional Requirements

The Elective Course Registration System must comply with university academic policies and regulations. Course registration rules, prerequisites, and seat limitations should follow the guidelines established by the university administration. Student information must also be handled in accordance with institutional data privacy policies.

5.3 Internationalization Requirements

The system should be designed to support future enhancements such as multi-language support if required by the institution. Although the initial version of the system will operate in English, the architecture should allow the addition of other languages without major modifications to the system.

5.4 Reusability Requirements

The system should be developed in a modular manner so that its components can be reused in other university systems, such as academic management systems or student portals. This approach will allow efficient development and easier integration with other institutional platforms in the future.

5.5 Maintainability Requirements

The system should be designed with clear documentation and modular architecture so that developers can easily update, maintain, and extend the system. Future improvements such as adding new courses, departments, or registration policies should be implemented without significantly affecting existing functionality.

Appendix A – Data Dictionary

Variable / Entity Name	Description	Type / Allowed States	Related Requirements
Student_ID	Unique identifier for each student (University Roll Number).	String (Unique)	F1, F6, F12
Student_Branch	The academic department the student belongs to (e.g., CSE, ECE).	Enumeration {Branch_1...Branch_5}	F2, F4
CGPA	Cumulative Grade Point Average used to determine academic merit for allocation.	Float (0.00 – 10.00)	F10, F11
Year_of_Study	The current academic year of the student (3rd or 4th).	Integer {3, 4}	F2, F10
Course_Code	Unique alphanumeric code assigned to each elective course.	String (Unique)	F3, F7
Course_Type	Categorization of the elective for rule enforcement.	{Branch_Elective, Humanities_Elective}	F4, F7
Seat_Capacity	The maximum number of students allowed to enroll in a course.	Integer (Constant per session)	F8, F13
Preference_Rank	The priority order assigned by the student to their selections.	Integer {1, 2, 3}	F4, F12
Submission_Time	The precise timestamp when the student clicks "Submit." Used as a tie-breaker.	DateTime	F6, F10

Priority_Score	The derived score calculated by the allocation algorithm.	Decimal (Calculated)	F10, F11
Allocation_State	The current status of a student's registration request.	{Pending, Allocated, Waitlisted}	F12, F14
User_Role	Permission level used to enforce security access (RBAC).	{Student, Administrator}	F1, F7

Appendix B - Group Log

Date	Time	Attendees	Primary Discussion & Outcomes
Feb 23, 2026	8:30 - 10:30 Am	All Members	**Project Initiation:** Officially selected the **Student Elective Registration System (SERS)** as the group project. Defined the primary goal of streamlining the Engineering School's elective process.
Feb 25, 2026	10:30 - 12:30 Am	All members	**Had an online meeting to finalise the project and project goal.**
Mar 2, 2026	8:30 - 10:30 AM	All Members	**Algorithm Design:** Discussed and finalized the **Priority Score** logic. Decided to rank students based on a combination of **CGPA, Year of Study, and Submission Timestamp** to ensure fairness.
Mar 9, 2026	8:30 - 10:30 Am	All Members	**Task Delegation & Tech Stack:** Divided the SRS sections among members. Finalized the use of **React** (Frontend), **Spring Boot** (Backend), and **PostgreSQL** (Database). Adopted the **COMET** methodology.
Mar 15, 2026	2:30 - 5:30 Pm	All Members	**Document Finalization:** Integrated the functional requirements (F1-F15) and non-functional requirements . Reviewed the interface requirements for both Students and Admins.

<rest will be added after discussion and after the end of the project>